00 01 02 03 04 05 06 07 08 09 0A 0B 0C **0D**

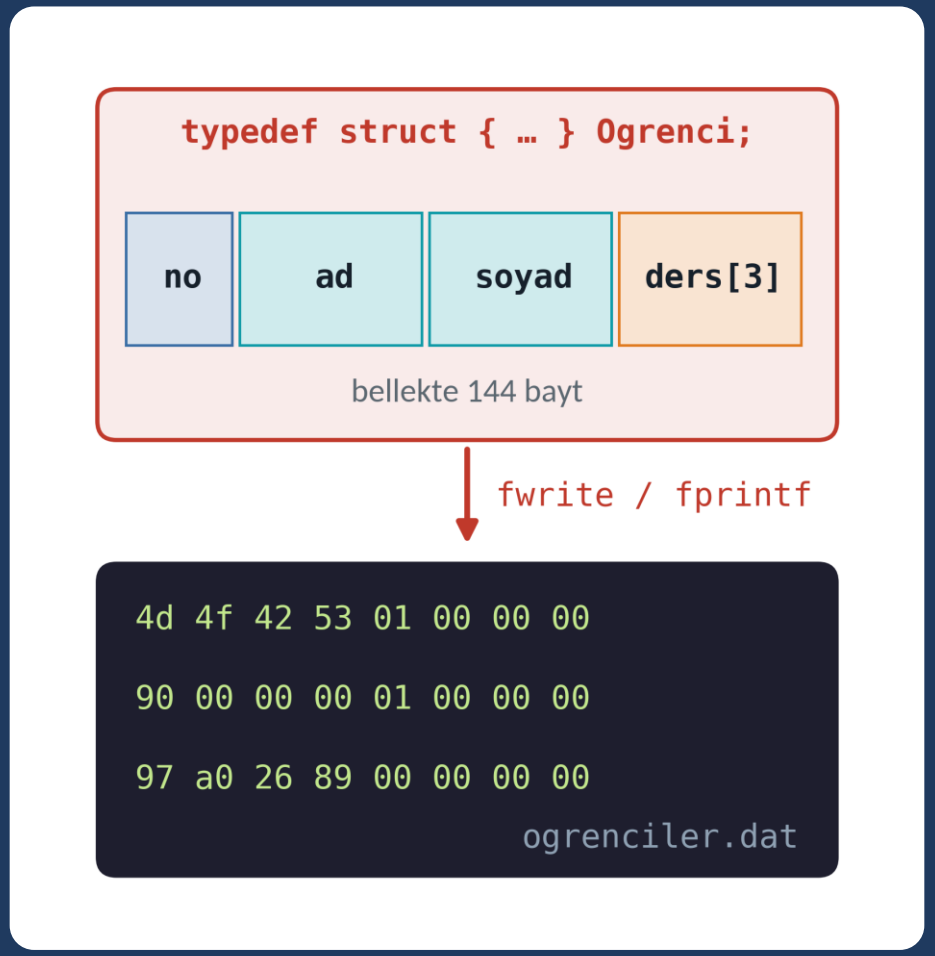
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 14

Yapılar, Dosyalar ve Bütünleştirici Proje: Gerçek Bir Program İnşa Etmek

struct ve typedef · -> ve const · enum ve union · hizalama ve dolgu · FILE * ve akışlar · tamponlama · metin/CSV ve ikili dosyalar · Mini-OBS v1.0 ve final projesi

MODÜL VI · METİN, KAYIT VE KALICILIK



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Bilgisayar ve Program	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** Operatörler ve İfadeler	**H5** Karar Yapıları	**H6** Döngüler I	**H7** Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Fonksiyonlar I	**H9** Kapsam, Yığın, Özyineleme	**H10** Diziler I	**H11** Diziler II: Matrisler	**H12** İşaretçiler ve Bellek	**H13** Karakter Dizileri	**H14** **Yapılar, Dosyalar, Final**
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GERİ ÇAĞIRMA · H13 · '\0' VE BOYUT + 1

`char ad[] = "MERT";` için `sizeof ad` ve `strlen(ad)` ne yazdırır?

quiz/H14_Q01_sonlandirici.c · satır 9–14

```c
int main(void)
{
    char ad[] = "MERT";
    printf("%zu %zu\n", sizeof ad, strlen(ad));
    return 0;
}
```

A) 5 4

B) 4 4

C) 4 5

D) 5 5

GERİ ÇAĞIRMA · H13

S2 (fgets ve '\n'), S3 (== ile strcmp) ve gerçek yanıtlar

S2 · quiz/H14_Q02 · klavyeden: Ali + Enter

```
11      char ad[16];
12      if (fgets(ad, sizeof ad, stdin) == NULL) {
13          return 1;
14      }
15      printf("%zu %s\n", strlen(ad), strcmp(ad, "Ali") == 0 ? "eşit" : "farklı");
```

S2 · GERÇEK ÇIKTI

```
$ ./q02
Ali
4 farklı
```

S3 · quiz/H14_Q03 · iki ayrı dizi

```
11      char a[] = "elma";
12      char b[] = "elma";
13      const char *p = a;
14      const char *q = b;
15      printf("%d %d\n", p == q, strcmp(p, q) == 0);
```

S3 · GERÇEK ÇIKTI

```
$ ./q03
0 1
```

- fgets **'\n'i saklar**: "Ali\n" 4 karakter → CSV satırlarında '\n' ve '\r' silinir
- `==` **adresleri**, `strcmp` **içeriği** karşılaştırır → iki yapı ise `==` ile **hiç** karşılaştırılamaz

BÜYÜK SORU

Haftanın Büyük Sorusu

Program kapandığında veriler nereye gider ve gerçek dünyadaki bir “kayıt” bellekte ve diskte nasıl temsil edilir?



Kayıt

Bir öğrencinin numarası, adı, notları: birbirine ait veriler tek bir tipte nasıl toplanır?



Kalıcılık

return 0’dan sonra ne kalır? RAM’deki her şey gider; yalnızca diske yazılmış baytlar kalır.



Gösterim

Aynı kayıt diskte iki biçimde: insanın okuduğu metin (CSV) ya da bellekteki baytlar (ikili).

*Haftanın sonunda yanıtı **onaltılı dökümle** ve **çalışan bir programla** göstereceğiz.*

GERÇEK DÜNYADAN

1997 · 2000

Yılı iki haneyle saklamanın bedeli: Y2K

- Onlarca yıl boyunca yıl **iki haneyle** saklandı (1997 → “97”): alan ve işletim maliyeti tasarrufu
- “00”: 2000 mi, 1900 mü? GAO’nun 1997 örneği: 1930 doğumlu kişinin yaşı 2000’de **–30**
- Aynı hesap `H14_K14_y2k_2038.c` ile: 1999’da 69, 2000’de **-30**
- Senato Y2K Komitesi (2000): maliyet ancak **tahmin** edilebildi; ABD için ≈ 100 milyar $, federal hükümet 8,34 milyar $, dünya için 320–500 milyar $ tahminleri

Willemssen (1997), GAO/T-AIMD-97-173 · United States Senate Special Committee on the Year 2000 Technology Problem (2000), S. Prt. 106-42

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 14.1	struct ile ilişkili verileri tek bir kayıt tipinde modeller; typedef, iç içe yapılar ve yapı dizileri kullanır.
ÖÇ 14.2	Yapıları fonksiyonlara değerle ve işaretçiyle aktarır; . ve -> operatörlerini doğru kullanır.
ÖÇ 14.3	Metin ve ikili dosyalarla kalıcı veri saklar (fopen/fclose, fprintf/fscanf, fgets/fputs, fwrite/fread) ve dosya hatalarını denetler.
ÖÇ 14.4	Dönem boyunca edinilen kavramları bütünleştirerek modüler, menü güdümlü ve verisini kalıcı olarak saklayan bir uygulama geliştirir, test eder ve sunar.

01

Yapılar: ilişkili verileri tek kayıtta toplamak

paralel dizilerden struct’a · atama ve karşılaştırma · typedef · iç içe yapı · yapı dizisi

Paralel dizilerden yapı dizisine

H10: üç paralel dizi

1046 yalnızca no[]'dan silindi (H10_K10)

	no[]	vize[]	final[]
0	1045	70	85
1	**1047**	**45**	**48**
2	**1048**	**90**	**95**

satırlar kaydı: 1047 artık 45 ve 48 ile eşleşiyor

(üç diziye aynı işlem: biri unutulursa veri bozulur)

`struct` →

H14: tek bir yapı dizisi

sinif[i] = sinif[i + 1]; (H14_K01)

```
[0]  no 1045   vize 70   final 85
[1]  no 1047   vize 90   final 95
[2]  no 1048   vize 55   final 60
```

bir kayıt = bir eleman:

alanları birbirinden ayrılamaz

Bir kayıt **tek eleman**: `sinif[i] = sinif[i + 1];` üç alanı birlikte taşır · v0.13'te paralel blok sayısı **6**

ÖRNEK 1 ★ · H14_01

struct tanımı, üye erişimi ve ilk değer: Örnek 1

H14_01_nokta_uzaklik.c · satır 12–15

```
12  struct Nokta {                  /* yeni bir TİP: iki double'lık kayıt */
13      double x;
14      double y;
15  };
```

H14_01_nokta_uzaklik.c · satır 26–31

```
26      struct Nokta a = {1.0, 2.0};            /* sırayla: x, y */
27      struct Nokta b = {.y = 6.0, .x = 4.0};  /* belirlenmiş ilk değer */
28
29      printf("A(%.1f, %.1f)  B(%.1f, %.1f)\n", a.x, a.y, b.x, b.y);
30      printf("|AB| = %.2f\n", uzaklik(a, b));
31      printf("sizeof(struct Nokta) = %zu bayt\n", sizeof(struct Nokta));
```

GERÇEK ÇIKTI

```
$ ./nokta_uzaklik      # gcc … -lm (sqrt)
A(1.0, 2.0)  B(4.0, 6.0)
|AB| = 5.00
sizeof(struct Nokta) = 16 bayt
```

- Tanım bir **tip** bildirir, bellek ayırmaz
- `struct Nokta`: tipin tam adı
- Üyeye **nokta** ile: `a.x`
- `{.y = 6.0, .x = 4.0}`: **belirlenmiş ilk değer** (C99)
- Adı geçmeyen üye: 0
- `}` sonrası `;` **zorunlu**

Atama kopyalar (diziler dahil), dizi ataması derlenmez

H14_K02_atama_kopya.c · satır 33–42

```c
33      Oyuncu a = {"CAN", 70};                 /* sırayla */
34      Oyuncu c = {.puan = 95, .ad = "EMRE"};  /* belirlenmiş ilk değer */
35      Oyuncu z = {0};                         /* bütün üyeler sıfır */
36
37      Oyuncu b = a;                /* yapı ataması: ad[] de KOPYALANIR */
38      b.ad[0] = 'X';
39      b.puan = 10;
40      printf("a.ad = %s, b.ad = %s\n", a.ad, b.ad);
41      printf("a.puan = %d, b.puan = %d\n", a.puan, b.puan);
42      printf("c = {%s, %d}, z = {\"%s\", %d}\n", c.ad, c.puan, z.ad, z.puan);
```

GERÇEK ÇIKTI

```
$ ./atama_kopya
a.ad = CAN, b.ad = XAN
a.puan = 70, b.puan = 10
c = {EMRE, 95}, z = {"", 0}
orta nokta = (2.0, 3.0)
```

H14_K03: `int a[3], b[3]; a = b;`
GCC: “assignment to expression with array type”

- `b = a`: **bütün** üyeler kopyalanır
- `b.ad` ayrı bir dizi: a değişmez
- Dizi atanamaz, ama yapı içindeki dizi **kopyalanır**
- Yapı **döndüren** fonksiyon: `Nokta orta_nokta(…)`

TAHMİN ET · YAPI ATAMASI

`Oyuncu b = a;` sonra b'nin puanı ve adının ilk harfi değişiyor. Program ne yazdırır?

quiz/H14_Q04_yapi_kopyasi.c · satır 12–20

```
12  int main(void)
13  {
14      Oyuncu a = {"CAN", 70};
15      Oyuncu b = a;
16      b.puan = 90;
17      b.ad[0] = 'J';
18      printf("%s %d / %s %d\n", a.ad, a.puan, b.ad, b.puan);
19      return 0;
20  }
```

A. `JAN 90 / JAN 90`

B. `CAN 70 / JAN 90`

C. `JAN 70 / JAN 90`

D. `CAN 90 / JAN 90`

Yapılar == ile karşılaştırılamaz; memcmp de güvenilmez

hata_avcisi/H14_HA1_yapi_esitlik.c (KASITLI HATA)

```
18      Nokta a = {1.0, 2.0};
19      Nokta b = {1.0, 2.0};
20      if (a == b) {                       /* HATA: yapılar için == yok */
21          printf("Aynı nokta\n");
22      }
```

GCC (HA1):
error: invalid operands to binary == (have ‘Nokta’ and ‘Nokta’)

H14_K04: DOLGUSU FARKLI İKİ KAYIT

```
$ ./karsilastirma
sizeof(Kayit) = 8 (üyeler 5 bayt)
alan alan: EŞİT, memcmp: FARKLI
a: 41 00 00 00 15 04 00 00
b: 41 FF FF FF 15 04 00 00
```

- `==` yalnızca aritmetik tipler ve işaretçiler için
- Çözüm: **alan alan** karşılaştıran fonksiyon
- `memcmp` dolgu baytlarını da karşılaştırır
- Dolgu baytlarının değeri **belirsiz**
- SEI CERT **EXP42-C**

typedef ve iç içe yapılar: Örnek 5



`o.dogum.yil → 2006` `(. soldan sağa zincirlenir: (o.dogum).yil)`

- `typedef`: tipe **yeni ad**, yeni tip değil
- Üye başka bir yapı olabilir: `o.dogum.yil`
- İç içe ilk değer: `{…, {14, 9, 2006}, …}`
- `sizeof(Tarih)` 12, `sizeof(Ogrenci)` 64
- Yıl **dört hane** (Y2K)

Yapı dizisi: en yüksek ortalamalı öğrenci (Örnek 6)

H14_06_en_yuksek_ortalama.c · satır 25–34

```c
int en_yuksek_indis(const Ogrenci a[], int n)
{
    int enb = 0;
    for (int i = 1; i < n; i++) {
        if (ortalama(a[i]) > ortalama(a[enb])) {
            enb = i;
        }
    }
    return enb;
}
```

GERÇEK ÇIKTI

```
$ ./en_yuksek_ortalama
1) 2301010045   79.00  AYŞE NUR
2) 2301010046   46.80  MEHMET ALİ
3) 2301010047   90.40  İSMAİL
4) 2301010048   88.35  CAN
En yüksek ortalama: İSMAİL (2301010047)
    90.40
sizeof sinif = 224, sizeof sinif[0] = 56
```

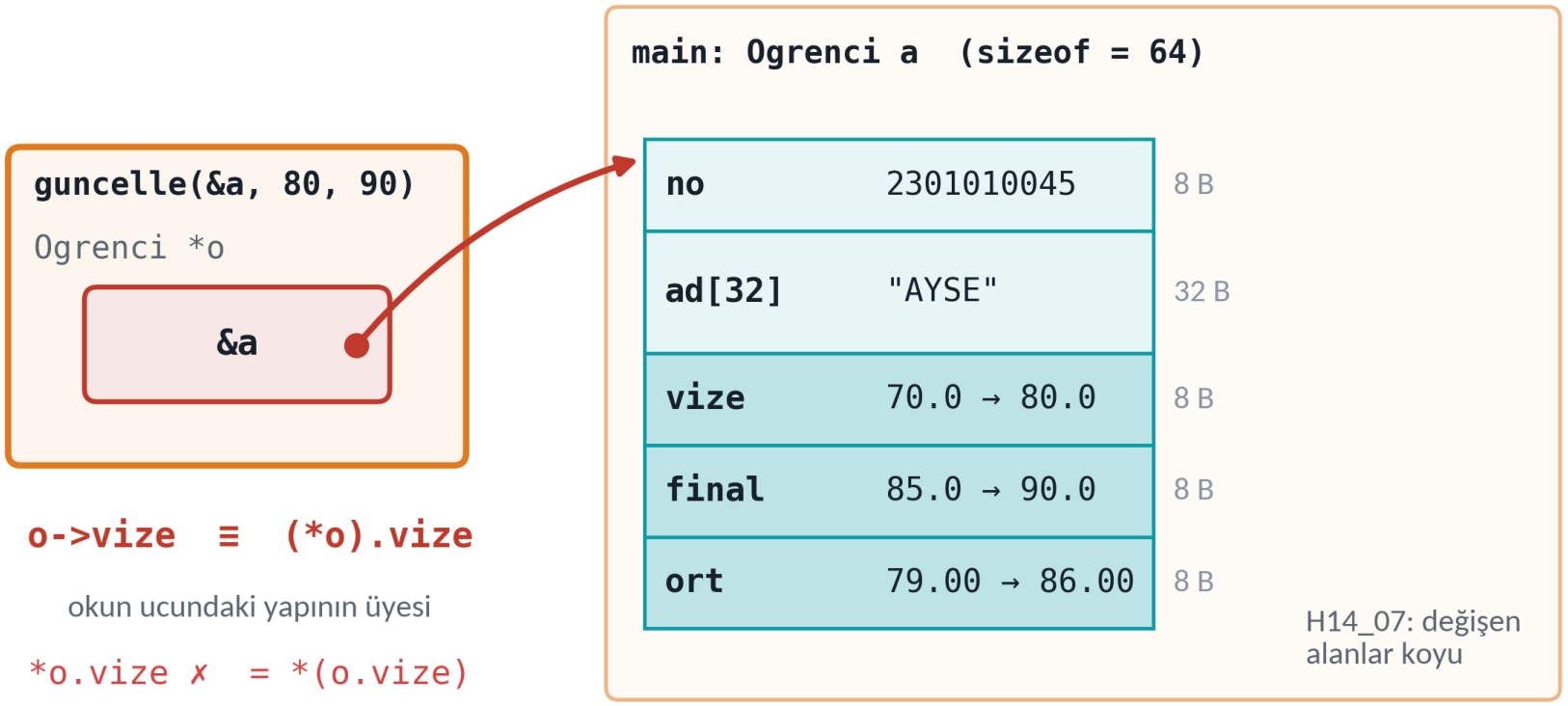
- H10'un deseni **değişmedi**
- İndis artık bir **kaydın tamamını** seçer
- `sizeof sinif / sizeof sinif[0]` = 224 / 56

02

İşaretçiler, enum ve dolgu

-> ile güncelleme · const ile salt okuma · enum ile menü · union · hizalama ve dolgu

Yapı işaretçisi ve ok operatörü



- `o->vize` ≡ `(*o).vize`
- `*o.vize` = `*(o.vize)`: **derlenmez**
- Solda yapı → **nokta**, işaretçi → **ok**
- Okun ucundaki kutu: main'in a'sı

Örnek 7: kaydı değiştiren fonksiyon adresi alır

H14_07_isaretci_guncelleme.c · satır 22–28

```c
void guncelle_kopya(Ogrenci o, double vize, double final)
{
    o.vize = vize;
    o.final = final;
    o.ort = 0.4 * vize + 0.6 * final;
    printf("  (kopyada ort = %.2f)\n", o.ort);
}
```

H14_07_isaretci_guncelleme.c · satır 31–36

```c
void guncelle(Ogrenci *o, double vize, double final)
{
    o->vize = vize;                 /* (*o).vize = vize; ile aynı */
    o->final = final;
    o->ort = 0.4 * vize + 0.6 * final;
}
```

- Değerle: `o` **64 baytlık kopya**; dönüşte yok olur
- İşaretçiyle: `o` main'in **kaydını** gösterir
- `ort` **türetilmiş** alan: üçü birlikte yazılır
- Mini-OBS: `ders_notu_ata` aynı karar

Örnek 7: const okuyucu, izleme ve gerçek çıktı

H14_07_isaretci_guncelleme.c · satır 39–43

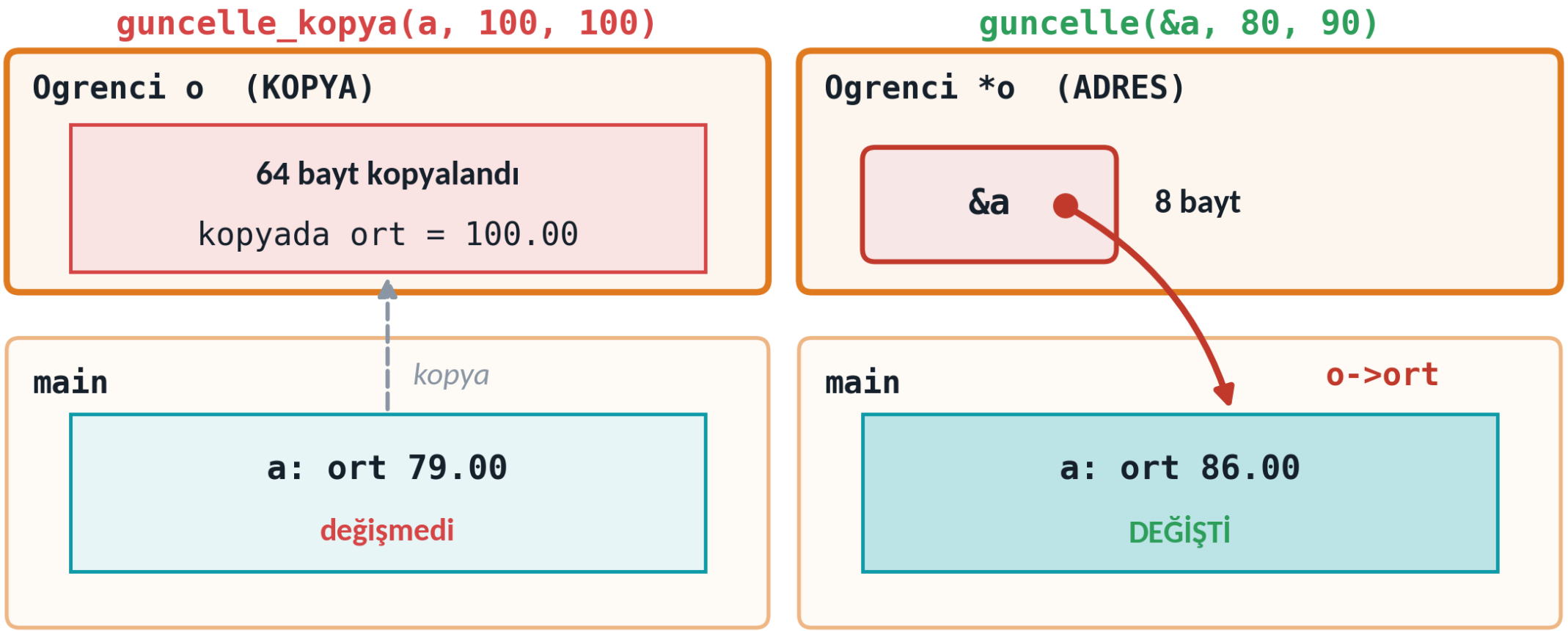
```
39  void yazdir(const Ogrenci *o)
40  {
41      printf("%lld %-5s vize %5.1f final %5.1f ort %5.2f\n",
42             o->no, o->ad, o->vize, o->final, o->ort);
43  }
```

Adım	a.vize	a.final	a.ort	kopya ort
başlangıç	70.0	85.0	79.00	—
guncelle_kopya	70.0	85.0	79.00	100.00
guncelle(&a,…)	80.0	90.0	86.00	—

GERÇEK ÇIKTI

```
$ ./isaretci_guncelleme
2301010045 AYSE  vize  70.0 final  85.0 ort 79.00
  (kopyada ort = 100.00)
2301010045 AYSE  vize  70.0 final  85.0 ort 79.00
2301010045 AYSE  vize  80.0 final  90.0 ort 86.00
p->ort = 86.00, (*p).ort = 86.00
Aktarılan: değerle 64 bayt, işaretçiyle 8 bayt
```

Değerle ve işaretçiyle aktarım: kopyanın bedeli



H14_K05: 4096 baytlık yapıyla 1 000 000 çağrı · değerle **0,034 s**, işaretçiyle **0,003 s** (bu makinede; oran önemli)

enum ile menü ve indis: Örnek 3

H14_03_enum_menu.c · satır 26–34

```
26  HarfNotu harf_bul(int ortalama)
27  {
28      for (HarfNotu h = AA; h < FF; h++) {    /* enum da bir tamsayıdır */
29          if (ortalama >= ALT_SINIR[h]) {
30              return h;
31          }
32      }
33      return FF;
34  }
```

GERÇEK ÇIKTI

```
$ ./enum_menu
78 -> BB (enum değeri 2)
AA=0 BA=1 BB=2 CB=3 CC=4 DC=5 DD=6 FD=7 FF=8
Geçersiz seçim: 7
Hoşça kalın.
```

- `typedef enum { MENU_HARF = 1, MENU_TABLO, MENU_CIKIS }`
- `case MENU_CIKIS:` (sihirli 3 yok)
- Sabitler `int`; aralık dışı değeri derleyici **engellemez**: `default` şart

MERAK KUTUSU

union: aynı baytlar, farklı yorumlar

H14_K06_union.c · satır 15–19

```c
typedef union {
    int i;
    float f;
    char c[8];
} Deger;
```

H14_K06_union.c · satır 23–29

```c
typedef struct {
    Tur tur;                    /* hangi üye geçerli? */
    union {
        int i;
        double d;
    } u;
} Sayi;
```

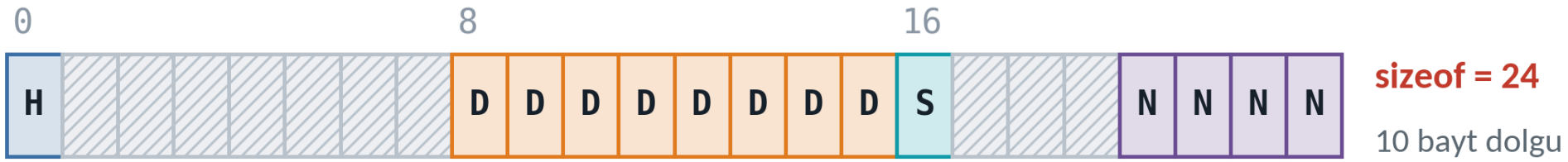
GERÇEK ÇIKTI

```
$ ./union_deneyi
sizeof(Deger) = 8
&d.i = 0x7ffc…, &d.f = 0x7ffc…
d.f = 1.0
1.0f'ın baytları int olarak:
    0x3F800000
tamsayı 42
ondalık 85.50
```

- Bütün üyeler **aynı adreste**
- Okunacak üye: **son yazılan**
- Güvenli kullanım: **etiketli union**

Kaputun altında: hizalama ve dolgu

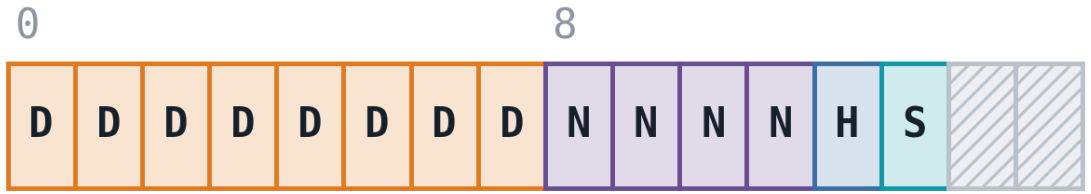
```
struct Kotu { char harf; double ort; char sube; int no; }
```



sizeof = 24

10 bayt dolgu

```
struct Iyi  { double ort; int no; char harf; char sube; }
```



sizeof = 16

2 bayt dolgu

H harf (char) · D ort (double) · S sube (char) · N no (int) · dolgu

Üyeler 14 bayt · `struct Kotu` **24** bayt · aynı üyeler büyükten küçüğe: **16** bayt · 1000 kayıtta fark 8 000 bayt

ÖRNEK 10 ★★★ · H14_10

Örnek 10: offsetof ile ölçmek

H14_10_dolgu_deneyi.c · satır 19–24

```c
struct Kotu {              /* küçük, büyük, küçük, orta */
    char harf;
    double ort;
    char sube;
    int no;
};
```

- **Kural:** hizalamaya göre büyükten küçüğe
- Mini-OBS'nin `Ogrenci`'si: 8 + 32 + 32 + 3 × 24 = **144**, dolgu 0

GERÇEK ÇIKTI · HER HARF BİR ÜYENİN BAYTI, '.' DOLGU

```
$ ./dolgu_deneyi
_Alignof: char 1, int 4, double 8
Kotu: harf 0, ort 8, sube 16, no 20  |H.......DDDDDDDDS...NNNN|
Iyi : ort 0, no 8, harf 12, sube 13  |DDDDDDDDNNNNHS..|
Üyelerin toplamı = 14 bayt
sizeof(struct Kotu) = 24, sizeof(struct Iyi) = 16
Dolgu: Kotu 10 bayt, Iyi 2 bayt
1000 kayıtlık dizi: 24000 ve 16000 bayt
```

TAHMİN ET · DOLGU

`struct Kayit { char c; int i; char d; };` x86-64 GCC'de `sizeof` kaçtır?

quiz/H14_Q06_dolgu.c · satır 7–17

```c
struct Kayit {
    char c;
    int i;
    char d;
};

int main(void)
{
    printf("%zu\n", sizeof(struct Kayit));
    return 0;
}
```

A 6

B 9

C 12

D 16

03

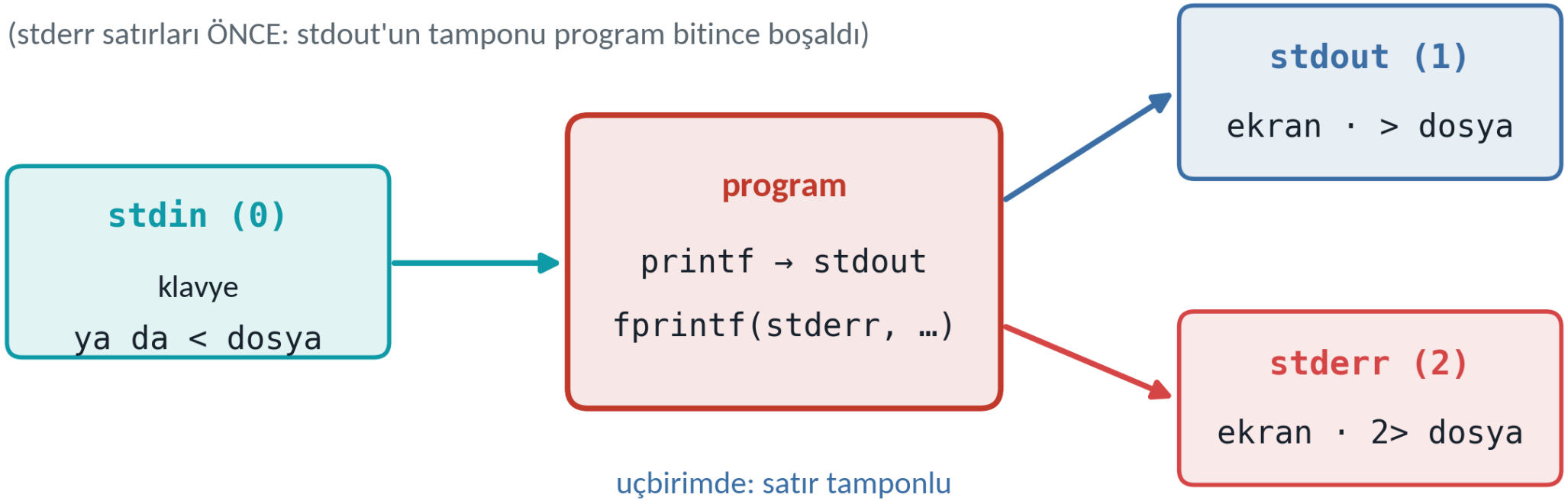
Dosyalar: verinin kalıcı olduğu yer

FILE * ve akışlar · fopen kipleri · hata denetimi · tamponlama · doğru okuma döngüsü · CSV · ikili dosya

FILE * ve üç standart akış

`./akislar > hepsi.txt 2>&1` → 2. satır, 4. satır, 1. satır, …

(stderr satırları ÖNCE: stdout'un tamponu program bitince boşaldı)



uçbirimde: satır tamponlu

dosyada: tam tamponlu

stderr: tamponsuz

`printf(…)` ≡ `fprintf(stdout, …)` · hata iletileri **stderr**'e · `>` stdout'u, `2>` stderr'i yönlendirir

fopen kipleri: ne açılır, ne silinir, yazma nereye gider?

kip	okur	yazar	dosya yoksa	dosya varsa	yazma konumu
r	✓	✗	NULL (hata)	içerik korunur	—
w	✗	✓	oluşturur	İÇERİK SİLİNİR	baştan
a	✗	✓	oluşturur	korunur	hep SONA
r+	✓	✓	NULL (hata)	korunur	baştan, üzerine
w+	✓	✓	oluşturur	İÇERİK SİLİNİR	baştan
a+	✓	✓	oluşturur	korunur	hep SONA
wx	✗	✓	oluşturur	NULL (hata)	baştan

wx: C11 ile gelen "özel oluşturma" kipi (w+x, wbx da olur): dosya varsa açılmaz.

b eklenirse (rb, wb, ab, r+b ...) İKİLİ kip: Windows'ta '\n' ↔ "\r\n" dönüşümü yapılmaz;

Linux'ta metin ve ikili kip aynıdır. Satırlar H14_K07'nin gerçek denemelerine göre.

TAHMİN ET · DOSYA AÇMA KİPLERİ

Dosyaya sırayla "w" ile "ABC", "w" ile "X", "a" ile "Y" yazılıyor. Son içerik?

quiz/H14_Q08_kipler.c · satır 7–14

```c
static void yaz(const char *kip, const char *metin)
{
    FILE *f = fopen("q08.txt", kip);
    if (f != NULL) {
        fputs(metin, f);
        fclose(f);
    }
}
```

A ABCXY

B XY

C XBC

D ABCY

Her dosya işlemi başarısız olabilir: NULL, perror, errno

H14_09_dosya_hatalari.c · satır 20–26

```
20      errno = 0;
21      FILE *f = fopen(yol, "r");
22      if (f == NULL) {
23          int hata = errno;                /* errno'yu HEMEN sakla */
24          perror(yol);                     /* "yol: ileti" → stderr */
25          fprintf(stderr, "  errno = %d (%s)\n", hata, strerror(hata));
26          return -1;
```

HA3: `fopen` dönüşü denetlenmedi, klasör yok

NULL akışa `fprintf`: **tanımsız davranış**

Gerçek: Segmentation fault, çıkış 139

ÖRNEK 9 · SIRADAN KULLANICIYLA GERÇEK ÇALIŞTIRMA

```
$ setpriv --reuid=65534 … ./dosya_hatalari puan.txt olmayan.txt gizli.txt klasor   # nobody
puan.txt: 3 satır
olmayan.txt: No such file or directory
  errno = 2 (No such file or directory)
gizli.txt: Permission denied
  errno = 13 (Permission denied)
klasor: Is a directory
4 dosyadan 3 tanesi okunamadı.
[çıkış kodu: 1]
```

Örnek 4: yaz, kapat, yeniden aç, satır satır oku

H14_04_metin_dosyasi.c · satır 32–43

```c
    f = fopen(DOSYA, "r");                  /* okumak için */
    if (f == NULL) {
        perror(DOSYA);
        return 1;
    }
    char satir[128];
    int sayac = 0;
    while (fgets(satir, sizeof satir, f) != NULL) {
        sayac++;
        printf("%d: %s", sayac, satir);     /* satir '\n' içerir */
    }
    fclose(f);
```

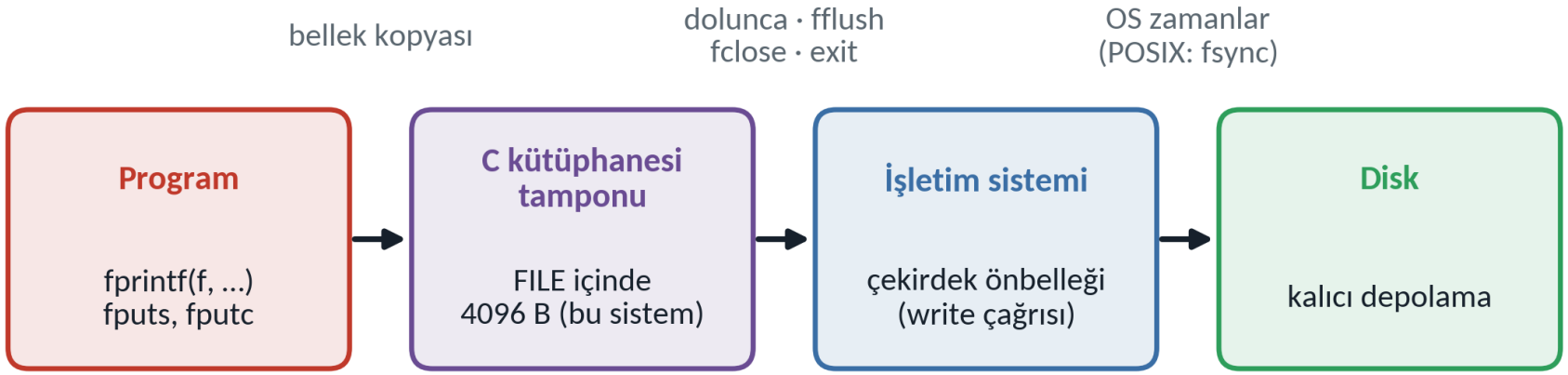
GERÇEK ÇIKTI

```
$ ./metin_dosyasi
1: 2301010045 70 85
2: 2301010046 45 48
3: 2301010047 88 92
3 satır okundu.
$ cat notlar.txt
2301010045 70 85
2301010046 45 48
2301010047 88 92
```

- Önce `w` ile `fprintf`/`fputs`, `fclose` **denetlenir**
- Döngü koşulu: **okumanın dönüşü**
- Satır '\n'i içerir: `printf("%d: %s")`

KAPUTUN ALTINDA

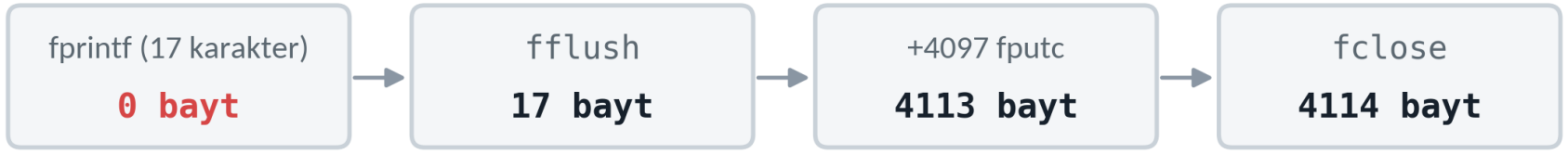
Kaputun altında: tamponlama



abort / çökme: tampondaki veri KAYBOLUR

(HA4: gunluk.txt 0 bayt kaldı)

H14_K09'un ölçümü (ikinci bir akışla dosyanın işletim sistemindeki boyutu):



17 karakterlik `fprintf` → **0 bayt** · `fflush` → **17** · 4 097. karakterde → **4 113** · `fclose` → **4 114**

Hata Avcısı 4: fclose unutuldu, program çöktü

hata_avcisi/H14_HA4_fclose_unutma.c (KASITLI HATA)

```
22      fprintf(f, "2301010045 notu girildi\n");
23      fprintf(f, "2301010046 notu girildi\n");
24      fprintf(f, "2301010047 notu girildi\n");
25      /* HATA: fclose(f) (ya da fflush(f)) yok */
26      printf("Üç kayıt yazıldı (sanıyoruz).\n");
27      fflush(stdout);
28      abort();                          /* beklenmedik bir çökme */
```

- Program “yazıldı” dedi, dosya **0 bayt**
- İşi biten akışı **hemen** kapat
- CERT **FIO42-C**

HATALI: VERİ TAMPONDA KALDI

```
$ ./ha4
Üç kayıt yazıldı (sanıyoruz).
Aborted                    ./ha4
çıkış durumu: 134
$ wc -c gunluk.txt
0 gunluk.txt
$ cat gunluk.txt
```

DÜZELTİLMİŞ: AKIŞ ÇÖKMEDEN ÖNCE KAPATILDI

```
$ ./ha4d
Üç kayıt yazıldı.
Aborted                    ./ha4d
çıkış durumu: 134
$ wc -c gunluk.txt
72 gunluk.txt
…
```

TAHMİN ET · DOSYA SONU DÖNGÜSÜ

3 baytlık dosyada (A, B, \n) `while (!feof(f)) { fgetc(f); n++; }` kaç tur döner?

quiz/H14_Q09_feof_sayac.c · satır 14–27

```
14      fputs("AB\n", f);
15      fclose(f);
16      f = fopen("q09.txt", "r");
17      if (f == NULL) {
18          return 1;
19      }
20      int n = 0;
21      while (!feof(f)) {
22          fgetc(f);
23          n++;
24      }
25      fclose(f);
26      remove("q09.txt");
27      printf("%d\n", n);
```

A 2

B 3

C 4

D 5

while (!feof(f)) karşıt örüntüsü ve doğru döngü

puanlar.txt: 70↵ 85↵ 90↵ (3 satır)

while (!feof(f)) ✗

tur	fgets döner	işlenen	feof
1	satır	70	false
2	satır	85	false
3	satır	90	false
4	NULL	90	TRUE

4 kayıt, toplam 335

son satır İKİ KEZ işlendi

while (fgets(…) != NULL) ✓

tur	fgets döner	işlenen	feof
1	satır	70	false
2	satır	85	false
3	satır	90	false
4	NULL	— (biter)	true

3 kayıt, toplam 245

dönüş değeri döngü koşulu

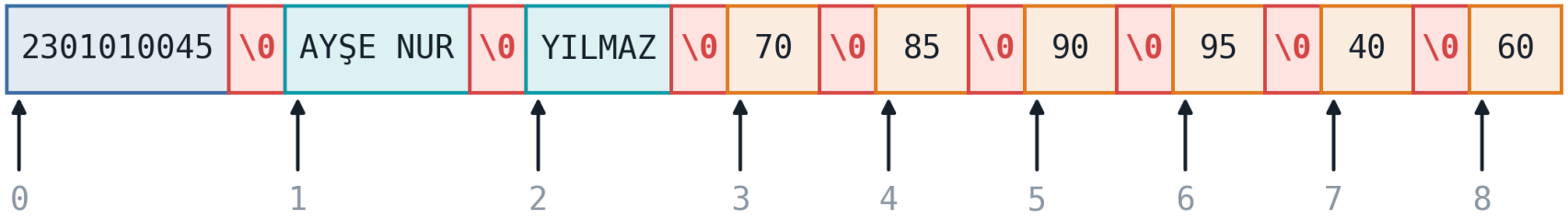
HA5: “4 kayıt, toplam 335” · düzeltilmiş: “3 kayıt, toplam 245” · `fscanf: == beklenen alan sayısı`

Bir CSV satırını alanlara ayırmak

1) fgets ile okunan satır (ornek_veri/ogrenciler.csv, 2. satır):

```
2301010045;AYŞE NUR;YILMAZ;70;85;90;95;40;60
```

2) alanlara_bol: her ';' yerine '\0' yazar; alan[i], i. alanın başını gösterir



3) her alan GÜVENİLMEZ girdi gibi denetlenip Ogrenci'ye yazılır:

```
alan[0]    → tamsayi_coz        → o.no
alan[1..2] → ad_normallestir    → o.ad, o.soyad
alan[3..8] → ondalik_coz        → ders_notu_ata(&o.ders[d], v, f)
```

biri geçersizse ya da numara mükerrerse: okuma durur, satır numarasıyla DOSYA_BICIM_HATASI

strtok **boş alanı yutar**: “strtok : 3 alan: [1] [ÇELİK] [70]” · alanlara_bol: “alanlara_bol: 4 alan: [1] [] [ÇELİK] [70]”

Örnek 11: satırı alanlara böl ve her alanı denetle

H14_11_csv_yukleme.c · satır 41–48

```c
41      char *alan[ALAN_SAYISI];
42      int n = 0;
43      for (char *p = strtok(satir, ";"); p != NULL; p = strtok(NULL, ";")) {
44          if (n == ALAN_SAYISI) {
45              return false;              /* fazla alan */
46          }
47          alan[n++] = p;
48      }
```

H14_11_csv_yukleme.c · satır 53–59

```c
53      o->no = strtoll(alan[0], &son, 10);
54      if (son == alan[0] || *son != '\0' || o->no <= 0) {
55          return false;
56      }
57      if (strlen(alan[1]) >= sizeof o->ad
58          || strlen(alan[2]) >= sizeof o->soyad) {
59          return false;                  /* '\0' için yer kalmalı */
```

- Fazla alan diziye **yazılmadan** reddedilir
- “Metnin **tamamı** sayı mı?” (H13)
- Ad: '\0' için yer kalmalı
- Satır sonu: `strcspn(satir, "\r\n")`
- Dizi doluysa `realloc` ile **2 katı** (H12_12)

Örnek 11: izleme, gerçek çalıştırma ve test

Satır	Sonuç	n	kap.
1	başlık: atla	0	0
2	yüklendi · realloc 0 → 4	1	4
3–4	yüklendi	3	4
5	“85,5”: atlandı	3	4
6	yüklendi	4	4
7	boş ad: atlandı	4	4
8	yüklendi · realloc 4 → 8	5	8
9–10	yüklendi	7	8

GERÇEK ÇIKTI (UYARILAR STANDART HATAYA)

```
$ ./csv_yukleme veri/sinif.csv
Uyarı: veri/sinif.csv satır 5 biçim hatası,
    atlandı
Uyarı: veri/sinif.csv satır 7 biçim hatası,
    atlandı
2301010045   79.00  AYŞE NUR YILMAZ
2301010046   46.80  MEHMET ALİ KAYA
2301010047   90.40  İSMAİL ÖZTÜRK
2301010049   76.00  ZEYNEP ÇELİK
2301010051   55.60  ŞULE DOĞAN
2301010052   36.00  ILGAZ ŞAHİN
2301010053   97.00  ELİF KARA
7 öğrenci yüklendi (kapasite 8), 2 satır
    atlandı.
Sınıf ortalaması: 68.69
```

Örnek 12: ikili dosyaya yapı dizisi yazma ve okuma

H14_12_ikili_dosya.c · satır 69–70

```
69      size_t n = fread(oku, sizeof oku[0], 10, f);    /* 10 istendi */
70      printf("fread %zu kayıt döndürdü\n", n);
```

H14_12_ikili_dosya.c · satır 74–79

```
74      long konum = 2 * (long)sizeof(Kayit);          /* 3. kaydın başı */
75      Kayit k;
76      if (fseek(f, konum, SEEK_SET) == 0 && fread(&k, sizeof k, 1, f) == 1) {
77          printf("3. kayıt (fseek %ld): %d %s %.2f\n", konum, k.no, k.ad,
78                 k.ort);
79      }
```

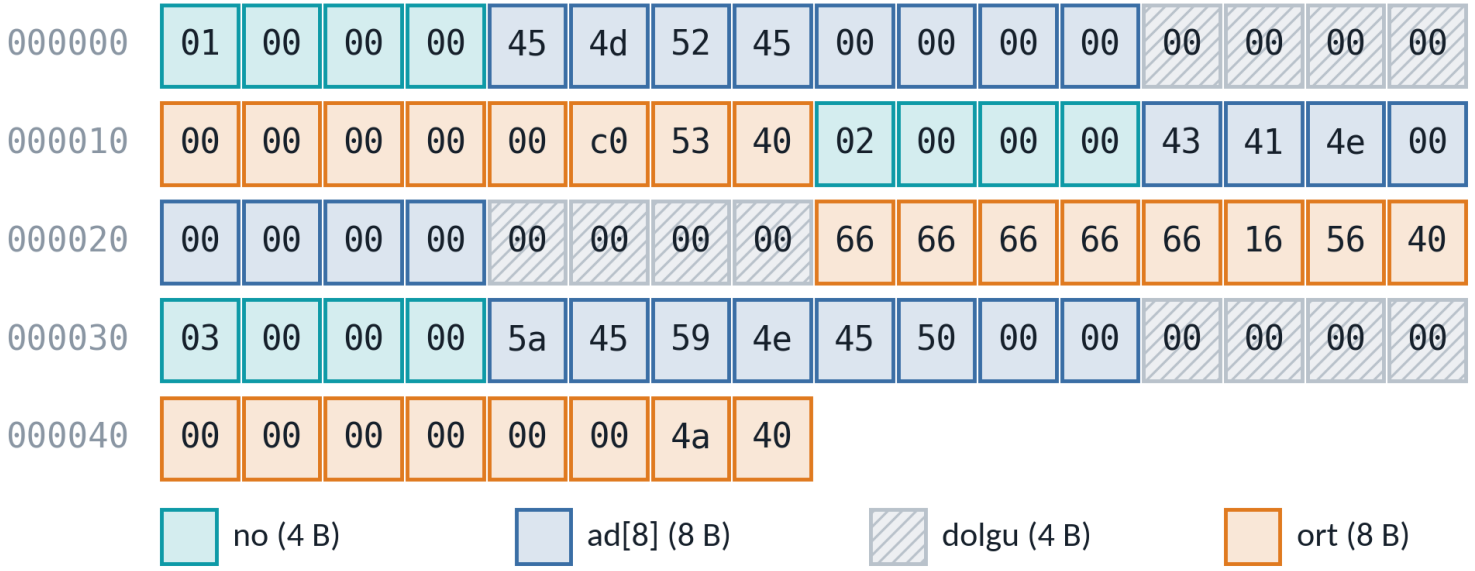
GERÇEK ÇIKTI

```
$ ./ikili_dosya
sizeof(Kayit) = 24, dosya = 72 bayt
fread 3 kayıt döndürdü
  1 EMRE   79.00
  2 CAN    88.35
  3 ZEYNEP 52.00
3. kayıt (fseek 48): 3 ZEYNEP 52.00
```

- `fwrite(yaz, sizeof yaz[0], 3, f)`: baytlar, **dolgu dahil**
- Dosya 72 bayt; aynı veri metin olarak 40
- `fread` **okuyabildiği** sayıyı döndürür
- Yazmadan önce `memset` (DCL39-C)

Onaltılı döküm: metin ve ikili dosyanın baytları

kayitlar.dat (fwrite, 3 × 24 bayt) — od -A x -t x1z




```
01 00 00 00  = 1        (int, küçük-uçlu: en anlamsız bayt önce)
00 00 00 00 00 c0 53 40  = 79.0  (double, IEEE 754)
```

```
kayitlar.txt (fprintf, 40 bayt): 1;EMRE;79.00↵ …
```

metin: rakamların karakter kodları (31 = '1', 3b = ';', 0a = '\n')

`od -A x -t x1z` (Linux) · `Format-Hex` (PowerShell) · ikili dosya **taşınabilir değildir**

TAHMİN ET · İKİLİ DOSYANIN BOYUTU

`typedef struct { int no; double ort; } Kayit;` üç kayıt `fwrite` ile yazılıyor. Dosya kaç bayttır?

quiz/H14_Q11_ikili_boyut.c · satır 8–21

```c
typedef struct {
    int no;
    double ort;
} Kayit;

int main(void)
{
    Kayit k[3] = {{1, 79.0}, {2, 46.8}, {3, 90.4}};
    FILE *f = fopen("q11.dat", "wb");
    if (f == NULL) {
        return 1;
    }
    fwrite(k, sizeof k[0], 3, f);
    long boyut = ftell(f);
```

A 48

B 36

C 24

D 12

Hata Avcısı: altı hata, gerçek iletiler

#	Hata	Gözlenen (gerçek)	Çözüm
HA1	a == b (iki yapı)	derleme hatası: invalid operands to binary ==	alan alan karşılaştır
HA2	p.ort, o->ort, *p.ort	“'p' is a pointer; did you mean to use '->'?”	p->ort, o.ort, (*p).ort
HA3	fopen dönüşü denetlenmedi	Segmentation fault, çıkış 139	if (f == NULL) { perror(yol); … }
HA4	fclose yok + abort()	gunluk.txt 0 bayt	akışı hemen kapat; fflush
HA5	while (!feof(f))	“4 kayıt, toplam 335” (doğrusu 3 / 245)	while (fgets(…) != NULL)
HA6	eklemek için "w"	iki eski kayıt silindi	"a" (ya da "wx")

04

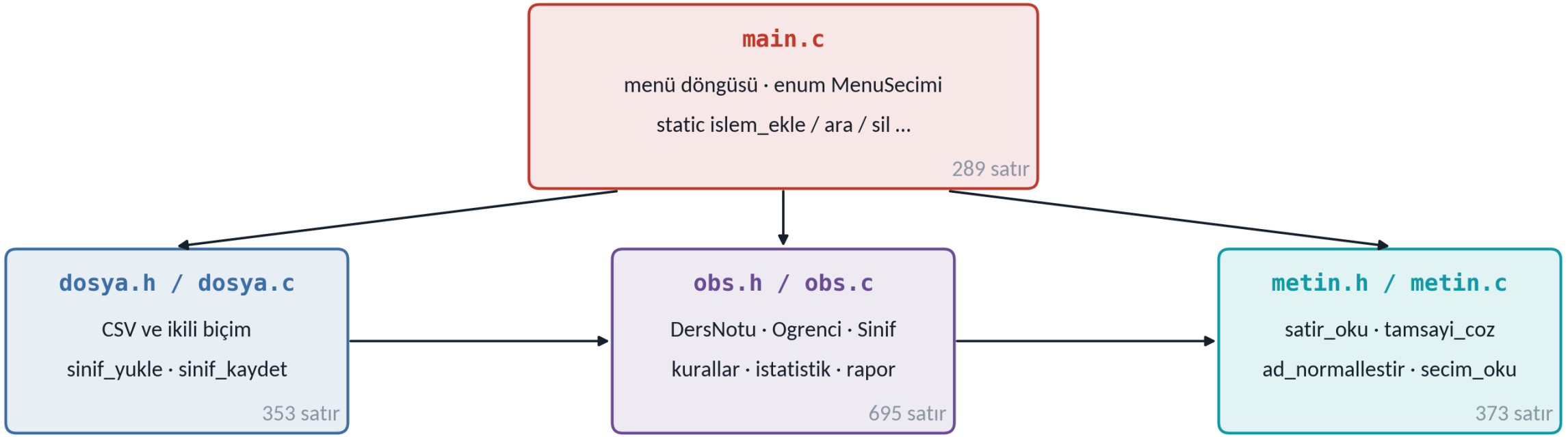
Mini-OBS v1.0 ve final projesi

dört modül · tek kayıt tipi · CSV ve ikili kalıcılık · test oturumları · final projesi, atölyeler ve rubrik

Mini-OBS v1.0: dört modül, tek kayıt tipi, kalıcı veri

ok: #include (bağımlılık)

test_obs.c: 99 doğrulama

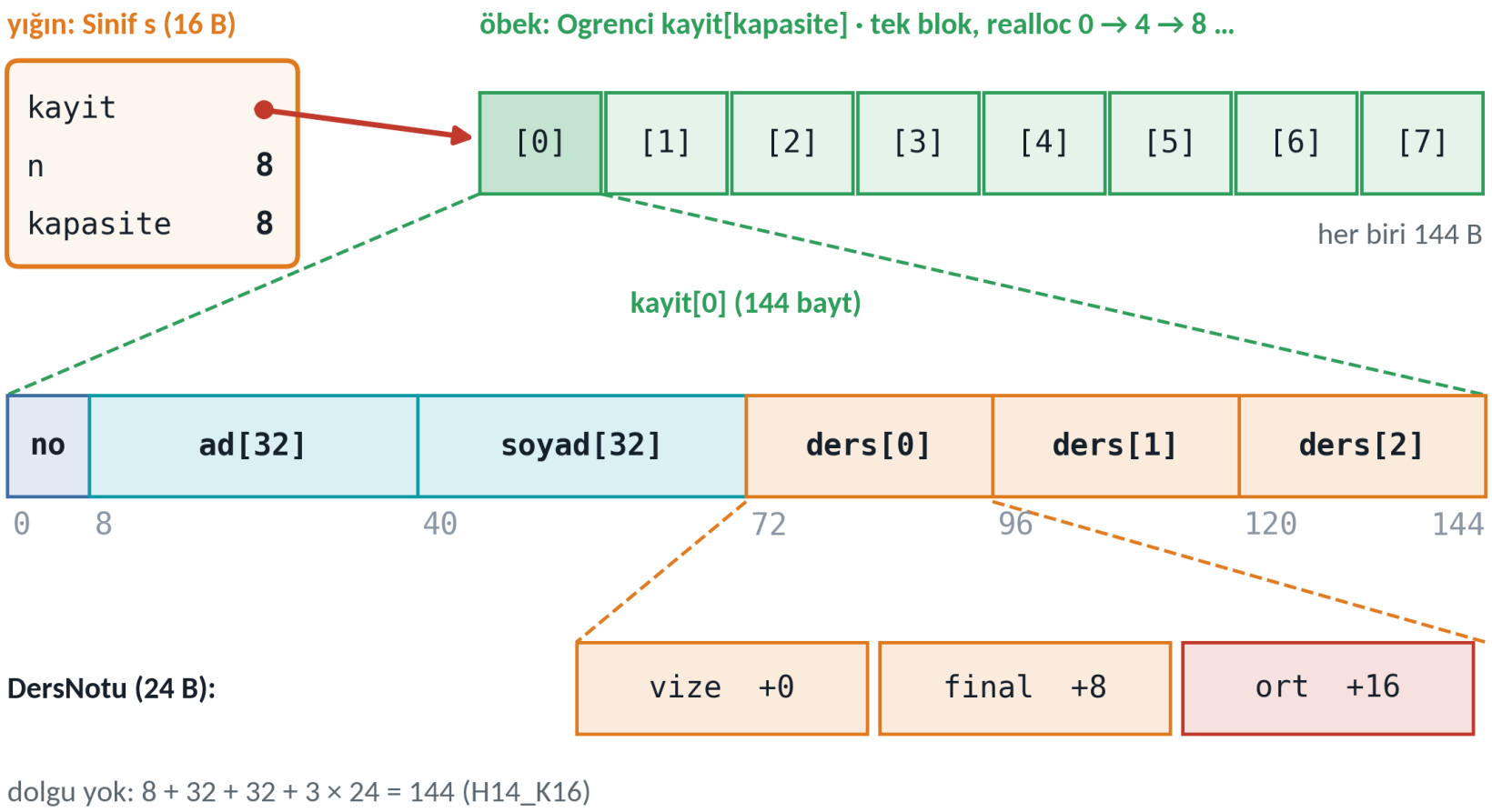


dosya.c, metin.h'yi de kullanır (sayı ve ad denetimi)

dış dünya: ogrenciler.csv/.dat ↔ dosya · klavye/ekran ↔ metin, main

- Hesap kuralları değişmedi (RTEÜ harf tablosu, %40 + %60, final barajı 50) · v0.13 ile istatistik ve harf dağılımı **birebir aynı** · 99 doğrulama: 37 kural + 22 metin + 24 yapı + 16 dosya

Veri modeli ve bellek düzeni

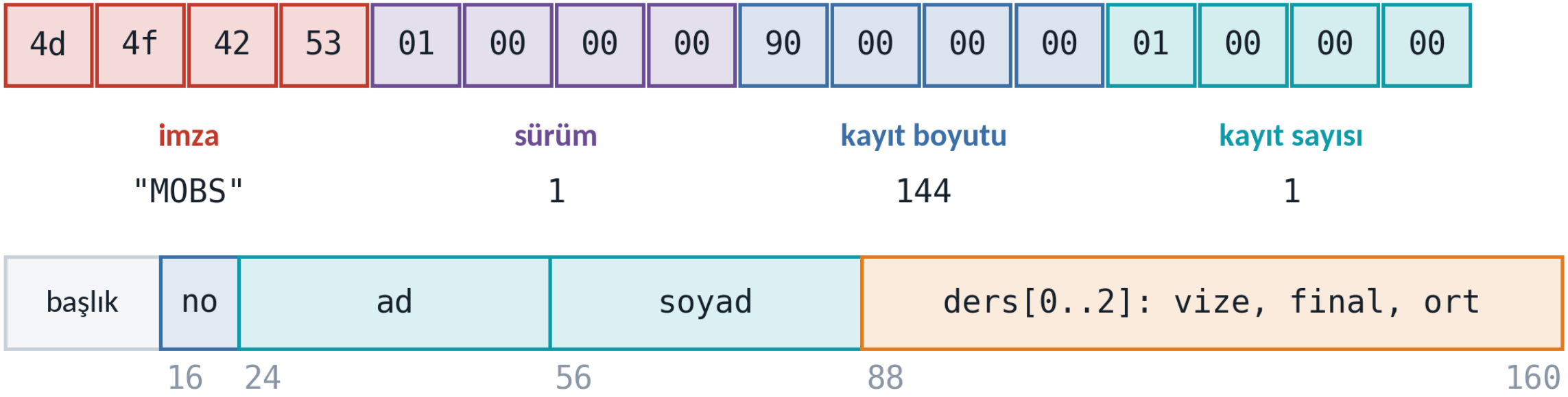


dolgu yok: 8 + 32 + 32 + 3 × 24 = 144 (H14_K16)

`Ogrenci` **144** bayt, dolgu 0 · `Sinif` 16 bayt (yığında) · kayıtlar öbekte **tek blok**, büyütme tek `realloc`

İkili dosya biçimi: 16 baytlık başlık + kayıtlar

t6.dat (160 bayt): 16 baytlık başlık + 1 × 144 baytlık Ogrenci



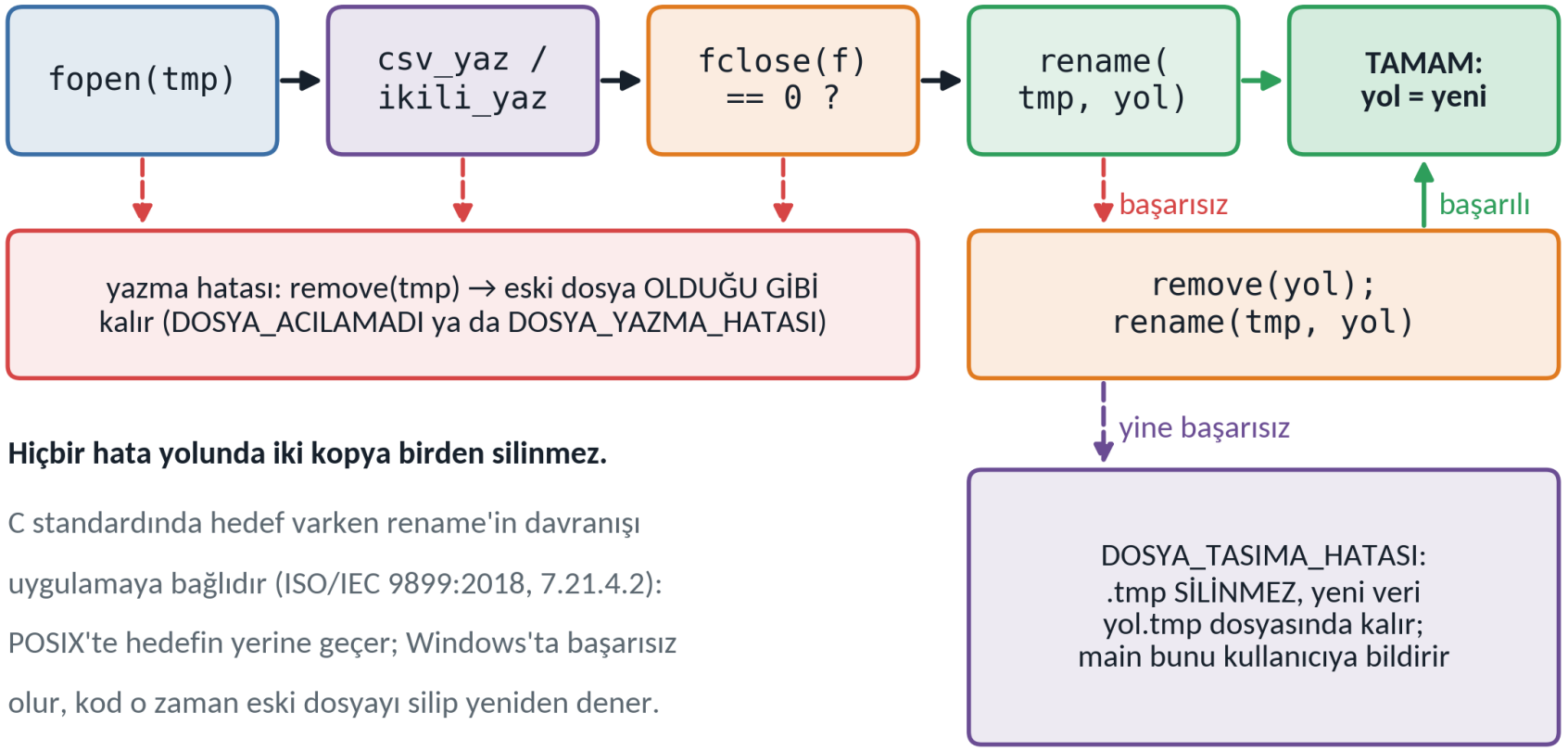
ad: 44 45 4e c4 b0 5a 00 00 … → "DENİZ" (İ = c4 b0), sonrası 00: bellek artığı yok

Başlık denetlenir (imza, sürüm, `sizeof(Ogrenci)`) · kayıtlar **tek tek** doğrulanır

MÜHENDİS GÖZÜYLE

Güvenli kaydetme: geçici dosya ve rename

sinif_kaydet(yol): önce yol.tmp'ye yaz, her adımı denetle, sonra yerine koy



Hiçbir hata yolunda iki kopya birden silinmez.

C standardında hedef varken rename'in davranışı uygulamaya bağlıdır (ISO/IEC 9899:2018, 7.21.4.2): POSIX'te hedefin yerine geçer; Windows'ta başarısız olur, kod o zaman eski dosyayı silip yeniden dener.

Yazma hatasında **eski veri yerinde kalır** · yerine konamazsa yeni veri **.tmp'de korunur** · hiçbir hata yolu iki kopyayı birden silmez

Test oturumları: canlı demo, bozuk dosya, bellek

T2 · ÖRNEK VERİ (GERÇEK OTURUM)

```
$ ./mini_obs t2.csv < testler/T2_ornek_veri.txt   # 2) Kayıtları listele
---------------------------------- KAYITLAR ----------------------------------
Sıra  Öğrenci no  Ad Soyad                        ALG        MAT        FIZ   YANO
   1  2301010045  AYŞE NUR YILMAZ          79.00 BB   93.00 AA   52.00 DC   2.91
   2  2301010046  MEHMET ALİ KAYA          46.80 FF   59.20 DC   66.00 CB   1.26
   3  2301010047  İSMAİL ÖZTÜRK            90.40 AA   69.00 CB   39.00 FF   2.29
   4  2301010048  CAN YILMAZ               88.35 BA  100.00 AA   30.00 FD   2.79
   5  2301010049  ZEYNEP ÇELİK             76.00 BB   63.00 CC   53.00 DC   2.21
…
```

T5 · BOZUK CSV: PROGRAM BAŞLAMAZ

```
$ ./mini_obs testler/bozuk.csv <
    testler/T5_bozuk_dosya.txt
Hata: testler/bozuk.csv okunamadı: dosya biçimi hatalı
    (satır 3)
çıkış kodu: 1
```

- **7 oturum** + kesik/imza/fazla bayt + kullanım hatası
- BOM'lu CSV okunur; taşıma hatasında yeni veri `.tmp`'de kalır
- `make asan`: rapor yok
- Valgrind T2: 9 allocs, 9 frees, 14,680 bytes allocated

Mühendis gözüyle: veri modelleme, dayanıklılık, güvenlik

Veri modeli uzun yaşar

- Yıl **dört hane**
- Türetilen değer **tek fonksiyonda**
- Kurala bağlı değer (harf) **saklanmaz**

Dosya güvenilmez girdidir

- Her alan klavye girdisi gibi **denetlenir**
- Bozuk dosya **yarım yüklenmez**
- ERR33-C, FIO34-C

Dayanıklı kaydetme

- Geçici dosya → `fclose` denetimi → `rename`
- FIO42-C: işi biten akışı kapat

Taşınabilirlik ve test

- CSV her yerde okunur; ikili **aynı derleyici/mimari**
- `tmpfile()` ile gidiş-dönüş testi
- EXP42-C, DCL39-C

Final projesi: seçenekler ve ortak asgari gereksinimler

Varsayılan: Mini-OBS v1.0

Kendi gerçeklemeniz; bu haftanın belirtimi ve testleri

Banka / ATM

hesaplar, işlemler, işlem geçmişi dosyada

Kütüphane ödünç

kitap ve üye kayıtları, ödünç/iade, gecikme

Stok ve satış

ürünler, satışlar, kritik stok raporu

Otopark / oyun / şifreleme

otopark yönetimi, skor tablosu dosyada metin oyunu, şifreleme aracı

Herkes için asgari

yapı dizisi · dinamik bellek · dosya kalıcılığı · çok dosyalı yapı · menü · girdi doğrulama

Atölyeler ve rubrik

1 · Kod incelemesi + eşli hata ayıklama

Eşler modül değiş tokuş eder; kontrol listesiyle satır numaralı bulgular · sürücü/gözcü, 7–8 dk'da rol değişimi

2 · Proje sunumu

8 dk sunum + 4 dk soru: problem, canlı demo, mimari, testler

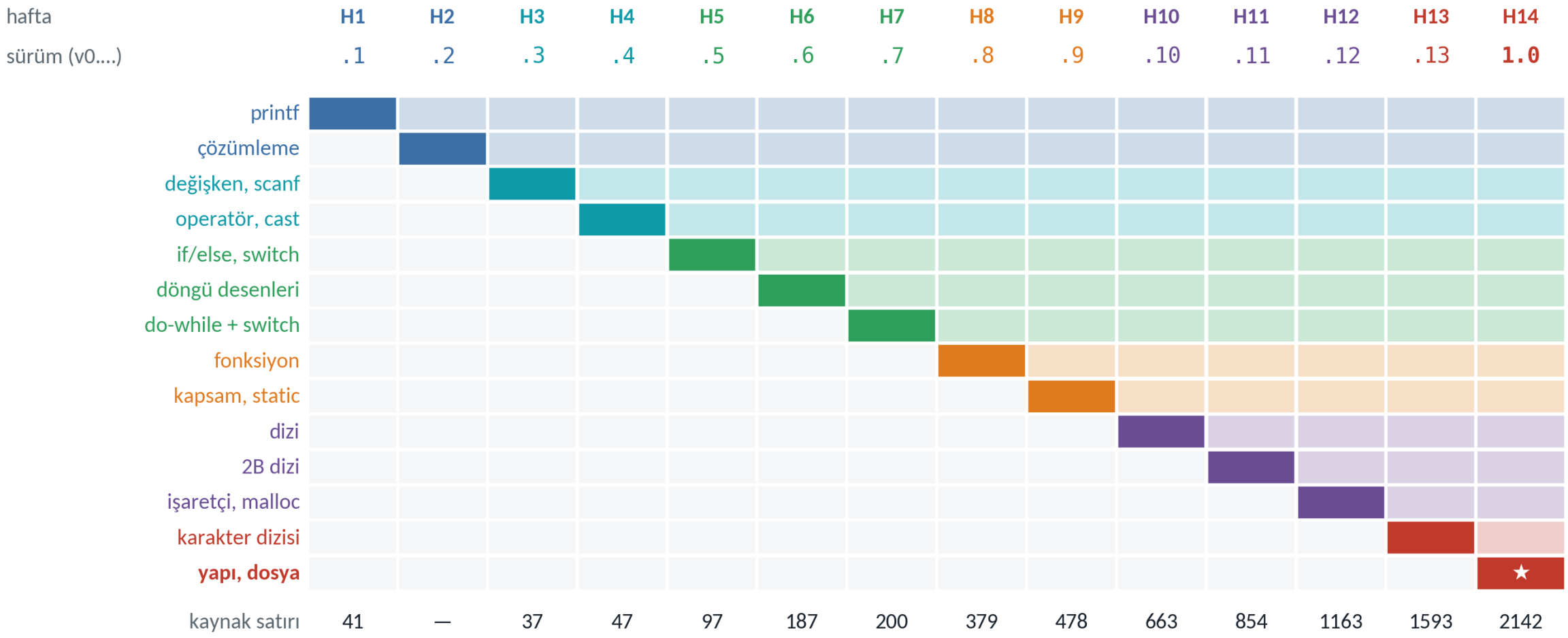
3 · Akran değerlendirmesi

İki ekip, beş ölçüt, 1–4 düzey; gözleme dayalı cümle + iki güçlü yön + bir öneri

Ölçüt	Ağırlık	Akranın sorusu
İşlevsel doğruluk	%30	Menü işlevleri ve sınır durumları doğru mu?
Algoritma ve tasarım	%20	Yapı dizisi, dinamik bellek, modüller anlamlı mı?
Kod kalitesi	%20	Adlar, biçim, uyarısız derleme, standart
Sağlamlık ve güvenlik	%15	Doğrulama, hata denetimi, sızıntı yok
Test ve sunum	%15	Senaryolar, canlı demo, gerekçeler

Düzeyler (1–4) ve puan aralıkları, akran formu ve kod inceleme listesi **final projesi yönergesinde**. Akran puanı geri bildirimdir; not öğretim üyesinin rubrik değerlendirmesidir.

Dönem haritası: v0.1'den v1.0'a



Her satır bir haftanın kavramı; koyu hücre kavramın Mini-OBS'ye girdiği sürüm, açık hücreler sonraki sürümlerde de kullanıldığını gösterir.

AP-2 KÖPRÜSÜ

AP-2 köprüsü: yapılar ve dosyalar nerede kullanılacak?

H14_K15_dugum.c · satır 14–22

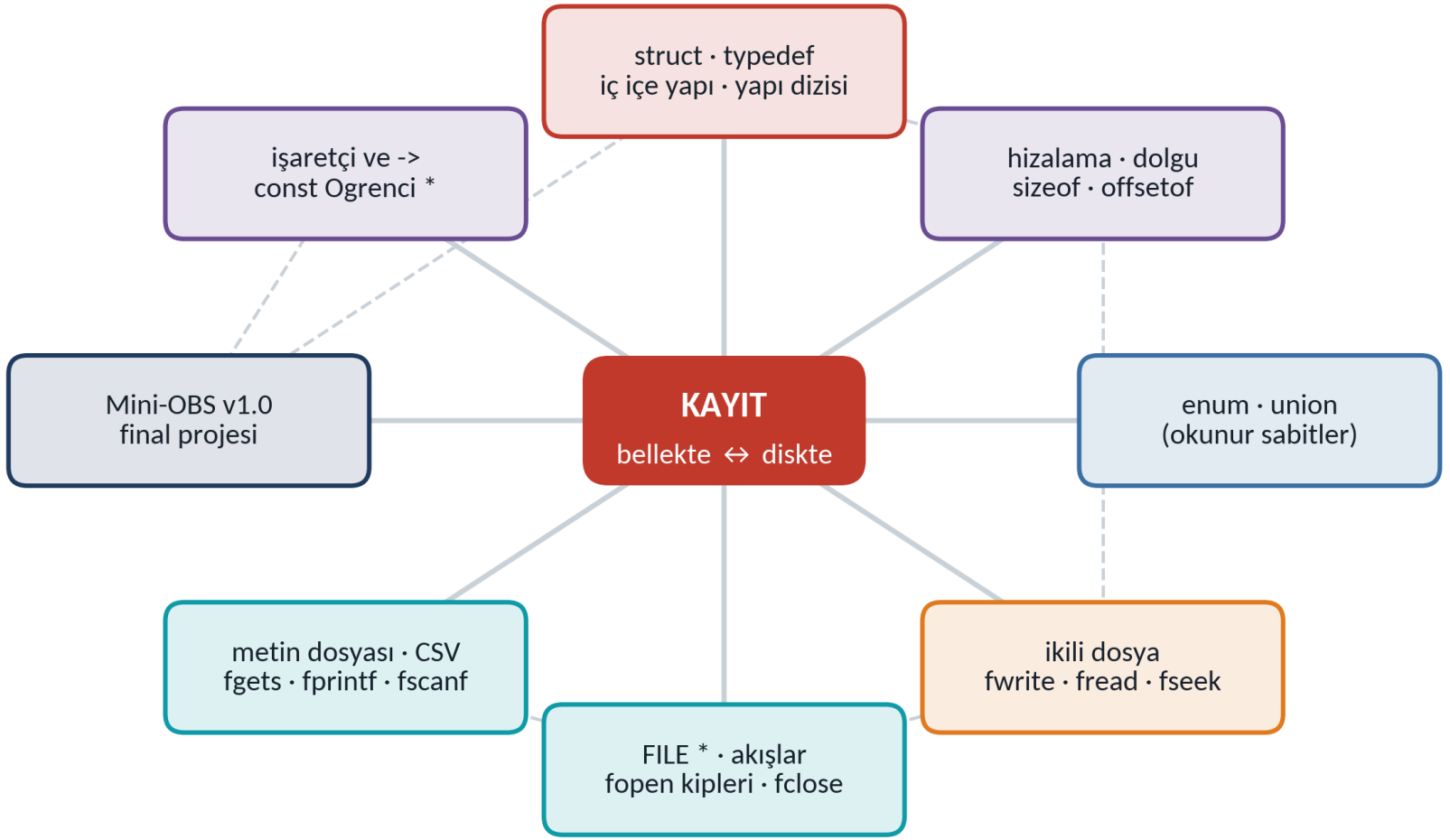
```c
14  struct Dugum {
15      int veri;
16      struct Dugum *sonraki;          /* bir sonraki düğümün ADRESİ */
17  };
18
19  int main(void)
20  {
21      struct Dugum b = {20, NULL};    /* NULL: zincirin sonu */
22      struct Dugum a = {10, &b};
```

GERÇEK ÇIKTI

```
$ ./dugum
sizeof(struct Dugum) = 16 bayt
a.veri = 10
a.sonraki->veri = 20
a.sonraki->sonraki == NULL
```

- Yapı dizisini **sıralamak** (ortalama, soyad): sıralama algoritmaları ve **kararlılık**
- Kendi tipinden **işaretçi**: bağlı liste ve dinamik veri yapıları
- Dosyadan büyük veri: algoritmaların **performansını ölçmek** (`clock()`, H7)
- Merak: 32 bitlik zaman damgası 2038-01-19 03:14:07 UTC'den bir saniye sonra taşar

Özet: kavram haritası



- **Yapı**: ilişkili veriler tek kayıtta; `.` ve `->`; atama kopyalar, `==` yok
- **Dolgu**: `sizeof` üyelerin toplamından büyük olabilir
- **Akış**: kip doğru seçilir; `fopen`, `fclose` ve okuma dönüşleri denetlenir
- **Tampon**: veri `fflush`/`fclose`/normal çıkışla işletim sistemine gider
- **CSV** taşınabilir; **ikili** hızlı ama taşınabilir değil
- Büyük Soru: kalıcı olan yalnızca **diske yazılmış baytlardır**

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. `struct Kayit { char c; double d; int i; };` x86-64'te kaç bayttır? Üyeleri yeniden dizerek kaça indirirsiniz?

2. Bir fonksiyon öğrencinin notunu **güncelleyecek**, başka biri yalnızca **yazdıracak**: iki parametre tipini yazın.

3. `while (!feof(f))` neden son kaydı iki kez işler? Doğru döngüyü yazın.